

Project Logbook

Project Title: SafeChat – A Private, Encrypted, Real-time Chat Application

Index

Sr.No	Week No	Contents	Date
1	Week 01	Project Group Formation	08/01/2026 – 15/01/2026
2	Week 02	Project Topic Finalization	19/01/2026 – 29/01/2026
3	Week 03	Requirement Analysis	29/01/2026 – 31/01/2026
4	Week 04	System Architecture Design	07/02/2026 – 18/02/2026
5	Week 05	Implementation Phase – I	19/02/2026 – 28/02/2026
6	Week 06	Design & Implementation	01/03/2026 – 09/03/2026
7	Week 07	Implementation Phase – II	10/03/2026 – 27/03/2026
8	Week 08	Implementation Phase – III	28/03/2026 – 01/04/2026
9	Week 09	System Testing	05/04/2026 – 27/04/2026
10	Week 10	Results & Analysis	28/04/2026 – 09/05/2026
11	Week 11	Final Report & Conclusion	09/05/2026 – 11/05/2026

Week 01: Project Group Formation

Date: 08/01/2026 – 15/01/2026

- Formed project group and assigned roles:
 - Frontend development (HTML, CSS, JavaScript)
 - Firebase integration and backend setup
 - Encryption and security module
 - Documentation and testing
- Brainstormed real-world issues like lack of privacy in messaging apps and need for secure communication.

Guide Interaction

Guide suggested selecting a socially relevant and impactful project.

Next Plan

Finalize project topic.

Week 02: Project Topic Finalization

Date: 19/01/2026 – 29/01/2026

- Explored domains:
 - Web Security
 - Real-time Communication
 - Cryptography
- Identified the problem: most simple chat apps store messages in plaintext on servers.
- Finalized project:

SafeChat – A Private, Encrypted, Real-time Chat Application

Guide Interaction

Guide suggested integrating browser-native encryption with a real-time database.

Next Plan

Requirement analysis.

Week 03: Requirement Analysis & System Study

Date: 29/01/2026 – 31/01/2026

- Defined system inputs:
 - Display Name

- Role selection (Sender / Receiver)
- Room Code (shared secret used for encryption)
- Defined outputs:
 - Real-time encrypted text messages
 - Encrypted file, image, and voice attachments
 - Typing indicator and online presence status
- Studied existing systems and identified limitations:
 - Plaintext message storage on servers
 - Requirement of account registration
 - Lack of browser-native encryption

Guide Interaction

Guide suggested focusing on client-side encryption and zero-account access model.

Next Plan

System architecture design.

Week 04: System Architecture Design

Date: 07/02/2026 – 18/02/2026

- Designed system modules:
 - Setup Screen (name, role, room code entry)
 - Encryption Module (Web Crypto API – AES-256-GCM)
 - Firebase Presence Module
 - Message Rendering Module
 - UI Shell (WhatsApp-style two-panel layout)
- Defined system flow:


```
User Input → Client Encryption → Firebase (Ciphertext) → Peer
      Decryption → Render
```
- Chose Firebase Realtime Database path structure:
 - safechat/rooms/{roomCode}/{messages, presence, typing}
- Designed onDisconnect() cleanup to prevent stale presence data.

Guide Interaction

Guide suggested using Firebase onDisconnect() for presence cleanup and structuring rooms to be self-contained.

Next Plan

Start implementation.

Week 05: Implementation Phase – I

Date: 19/02/2026 – 28/02/2026

- Finalized tech stack:
 - Frontend: HTML5, CSS3, Vanilla JavaScript
 - Backend / Realtime DB: Firebase Realtime Database
 - Encryption: Web Crypto API (AES-256-GCM)
- Set up Firebase project and obtained config credentials.
- Implemented basic HTML structure:
 - setupScreen div for room entry
 - chatApp div for the main chat interface
- Built connectToRoom() function and room code generator.

Guide Interaction

Guide suggested focusing on client-side key derivation to avoid transmitting the encryption key.

Next Plan

Feature extraction and encryption module implementation.

Week 06: Design & Implementation

Date: 01/03/2026 – 09/03/2026

- Implemented AES-256-GCM encrypt() and decrypt() using Web Crypto API.
- Key derivation:
 - Room Code padded/truncated to 32 bytes
 - Imported as raw CryptoKey for AES-GCM
 - Fresh 12-byte random IV generated per message
- Implemented full CSS design:
 - WhatsApp-inspired two-panel layout (left panel + right panel)
 - Chat bubbles, typing indicator, emoji tray, waiting overlay
- Built sendMessage() and Firebase child_added listener.

Guide Interaction

Guide suggested improving UI clarity and ensuring the encryption status banner is always visible.

Next Plan

Implement file attachment and voice recording features.

Week 07: Implementation Phase – II

Date: 10/03/2026 – 27/03/2026

- Implemented file attachment support:
 - FileReader converts file to base64 Data URL
 - JSON payload encrypted (type, fileName, fileType, dataUrl)

- Receiver decrypts and renders image, audio, or file element
- Implemented voice note recording:
 - navigator.mediaDevices.getUserMedia() for microphone access
 - MediaRecorder API with audio/webm MIME type
 - Start/stop toggle with UI feedback
- Implemented real-time typing indicator:
 - typRef.set() on input, auto-removed after 2500 ms of inactivity
 - Firebase typing listener filters out own typing events
- Implemented peer presence detection via syncPresence().

Guide Interaction

Guide suggested testing the voice note pipeline on multiple browsers due to codec differences.

Next Plan

UI polish, emoji tray, and logout functionality.

Week 08: Implementation Phase – III

Date: 28/03/2026 – 01/04/2026

- Completed emoji tray with 12 emojis and click-outside dismissal.
- Implemented auto-resize textarea and Enter/Shift+Enter key handling.
- Implemented Leave button:
 - Removes presence and typing records from Firebase
 - Detaches all Firebase listeners
 - Resets entire UI back to setup screen
- Added seen key deduplication (seenKeys Set) to prevent duplicate renders.
- Connected chat list preview to latest message content and timestamp.

Guide Interaction

Guide suggested the seenKeys deduplication approach and ensuring the leave sequence removes all Firebase listeners.

Next Plan

System testing.

Week 09: System Testing

Date: 05/04/2026 – 27/04/2026

- Tested:
 - Cross-browser compatibility (Chrome, Firefox, Edge, Safari)
 - Cross-device scenarios (Desktop-Desktop, Desktop-Mobile, Mobile-Mobile)
 - Encryption correctness with correct and wrong room codes

- Firebase onDisconnect() presence cleanup on browser close
- Used real encrypted message exchanges for end-to-end validation.
- Fixed Safari MediaRecorder issue – added graceful fallback message.

Guide Interaction

Guide suggested testing the wrong room code scenario to ensure the fallback error message is user-friendly.

Next Plan

Result analysis.

Week 10: Results & Analysis

Date: 28/04/2026 – 09/05/2026

- SafeChat successfully delivered all planned features.
- Encryption validated: no plaintext stored or transmitted outside the browser.
- System performance observed:
 - Message latency under 300 ms on average
 - File attachments up to 5 MB functional
 - Voice notes working on Chrome, Firefox, and Edge
- Planned future enhancements:
 - ECDH-based key exchange for stronger security
 - Message expiry and disappearing messages

Guide Interaction

Guide suggested adding analytical insights comparing SafeChat's security model with existing messaging apps.

Next Plan

Final documentation.

Week 11: Final Report & Conclusion

Date: 09/05/2026 – 11/05/2026

- Completed documentation and project report.
- Prepared final presentation and demo.
- Project implemented successfully.

Guide Interaction

Guide approved final submission.

Next Plan

Project evaluation.

Project Description

SafeChat

SafeChat is a real-time, peer-to-peer encrypted web chat application that allows two users to communicate securely using AES-256-GCM encryption. The application is built entirely on the client side using HTML5, CSS3, and Vanilla JavaScript, with Firebase Realtime Database as the signalling and message relay backend.

Features

- Takes display name, role, and room code as input
- Uses Web Crypto API to perform:
 - AES-256-GCM encryption with per-message random IV
 - Client-side key derivation from the shared Room Code
- Uses Firebase Realtime Database for message relay
- Supports multiple message types:
 - Text messages
 - Image attachments
 - Audio / voice notes
 - File attachments
- Provides real-time typing indicator and online presence status
- No messages stored in plaintext – all content encrypted before transmission

Additional Functionalities

- Emoji picker tray with 12 emojis
- Random room code generator
- Automatic peer disconnect detection and UI feedback
- Copy room code button for easy sharing
- WhatsApp-inspired responsive UI design

Future Scope

- ECDH key exchange for mathematically provable end-to-end security
- Message expiry and disappearing messages feature
- Multi-party group chat rooms
- Progressive Web App (PWA) packaging for mobile installation
- Push notifications via Firebase Cloud Messaging
- Server-side message TTL cleanup using Firebase Cloud Functions

Team Members

- Devansh Yogesh Chaudhari
- Sakshi Devidas Shimpi

- Shubham Dipak Wagh
- Utkarsh Ratilal Mahajan

Guide

Prof. Priyanka Lanjewar